

* Basics Numpy Operations:-

import numpy as np.

1) arr = np.array([10, 20, 30, 40]) # printing an array
print(arr)

2) arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # printing shape of an array.
print(arr2)
print(arr2.shape)

3) a = np.zeros((2, 3)) # print an ~~array~~^{matrix} having all values to 0
print(a)

4) a = np.ones((2, 3)) # print an matrix having all values to 1.
print(a)

5) a = np.arange(1, 10, 2) # creates a numpy array with a range of values.
print(a)

6) arr = np.array([10, 20, 30, 40, 50]) # Array indexing & slicing.
print(arr[0])
print(arr[1:4])
print(arr[-1])

7) a = np.array([[1, 2],
 [3, 4]])
b = np.array([[4, 3],
 [2, 1]])

Addition, subtraction, multiplication,
Transpose.

print(a+1)

print(b-2)

print(a+b)

print(a @ b)

print(a.T)

8) print(np.sum(arr))
print(np.sqrt(arr))
print(np.mean(arr))
print(np.max(arr))
print(np.min(arr))
print(np.std(arr))

basic mathematical operations

9) arr = np.array([1, 2, 3, 4, 5, 6]) # reshaping the array.
np.reshape(arr, (1, 2))

```
10) a = np.array([15, 6], [7, 8])  
res = a.flatten()  
print(res).
```

* Basic Operations of Pandas :-

1) import pandas as pd

```
s = pd.Series({'Amit': 85, 'Riya': 90, 'Karan': 78})
```

```
print(s)
```

```
print(s[1])
```

```
print(s['Riya'])
```

```
print(s.max())
```

```
print(s.min())
```

```
print(s.mean())
```

```
print(s.std())
```

2) data = {'Name': ['Amit', 'Riya', 'Karan'], 'Marks': [85, 90, 78],
 'City': ['Delhi', 'Mumbai', 'Pune']}

```
df = pd.DataFrame(data, index=['R1', 'R2', 'R3'])
```

```
print(df)
```

```
print(df.shape)
```

```
print(df.columns)
```

```
print(df.index)
```

```
print(df.head())
```

```
print(df.tail())
```

```
print(df.info())
```

```
print(df.loc[1])
```

```
print(df.iloc[1])
```

```
print(df.loc[0:2])
```

```
print(df.iloc[0:2])
```


4.	Name	Marks
R0	karan	10
R1	Aditi	20
R2	Hanshit	30
R3	Neha	25

```
new_rows = pd.DataFrame({'Name': ['Vijay', 'Rohit'],
                          'Marks': [30, 40]})
```

```
pd.concat([df, new_rows], axis=0)
pd.concat([df, new_rows], ignore_index=True)
df.drop(1, axis=0)
df.drop([1, 3], axis=0)
df.sort_values(by='Marks')
df.sort_values(by='Marks', ascending=False)
df.rename(columns:
           {'Marks': 'Score'})

df.isnull()
df.dropna()
```

Graph

* import seaborn as sns

```
x = np.array([5, 10, 15, 20, 24, 30])
```

```
y = np.array([10, 20, 30, 40, 50, 60])
```

```
plt.figure(figsize=(6, 4))
```

```
plt.plot(x, y, colour='blue', marker='o', line_style='--')
```

```
plt.title("Line graph of X vs Y")
```

```
plt.xlabel("X values")
```

```
plt.ylabel("Y values")
```

```
plt.grid(True)
```

```
plt.show()
```

1st January 2026

Create a dataset for Healthcare.

A. Import pandas as pd

data = {

"Patient-Id": [101, 102, 103, 104, 105],

"Name": ["Vanshika", "Atharva", "Agastya", "Vansh", "Abir"],

"Age": [19, 15, 25, 24, 26],

"Gender": ["F", "M", "M", "M", "M"],

"Blood Group": ["A+", "B+", "O+", "AB+", "O-"],

"Disease": ["Diabetes", "BP", "Heart", "Asthma", "Dengue"],

"Height-cm": [170, 160, 175, 158, 160],

"Weight-kg": [66, 60, 70, 55, 72]

}

df = pd.DataFrame(data)

1 print(df)

2 df.head()

3 df.tail()

4 df.info()

5 df.describe()

Output:- (for 1, 2, 3).

Patient-ID	Name	Age	Gender	Blood Group	Disease	Height-cm	Weight-kg
101	Vanshika	19	F	A+	Diabetes	170	66
102	Atharva	15	M	B+	BP	160	60
103	Agastya	25	M	O+	Heart	175	70
104	Vansh	24	M	AB+	Asthma	158	55
105	Abir	26	M	O-	Dengue	160	72

Output (4)

```
<class 'Pandas.core.frame.DataFrame'>
```

Range Index 5 entries, 0 to 4

Data columns (total 8 columns):

Patient_ID	int64
Name	object
Age	int64
Gender	object
Blood-Group	object
Disease	object
Height-cm	int64
Weight-kg	int64

```
5) df[df["Age"] > 20]
   df[df["Disease"] == "Dengue"]
```

Output -

Patient-Id	Name	Age	Gender	Blood-group	disease	Height-cm	weight
103	Agastya	25	M	O+	Heart	175	70
104	Vansh	24	M	AB+	Asthma	150	55
105	Abir	26	M	O-	Dengue	160	72

Patient-Id	Name	Age	Gender	Blood-group	Disease	Height-cm	weight
105	Abir	26	M	O-	Dengue	160	72

6) import numpy as np

```
ages = np.array(df["Age"])
```

```
weights = np.array(df["weight-kg"])
```

```
print("Avg Age:", np.mean(ages))
```

```
print("Max Weight:", np.max(weights))
```

```
print("Min weight:", np.min(weights))
```

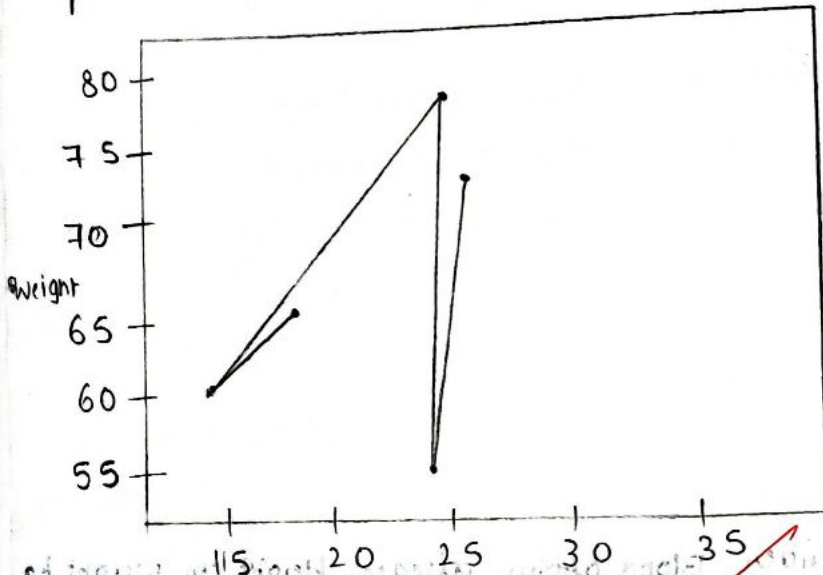
Output -

Avg Age: ~~27~~ 21.0

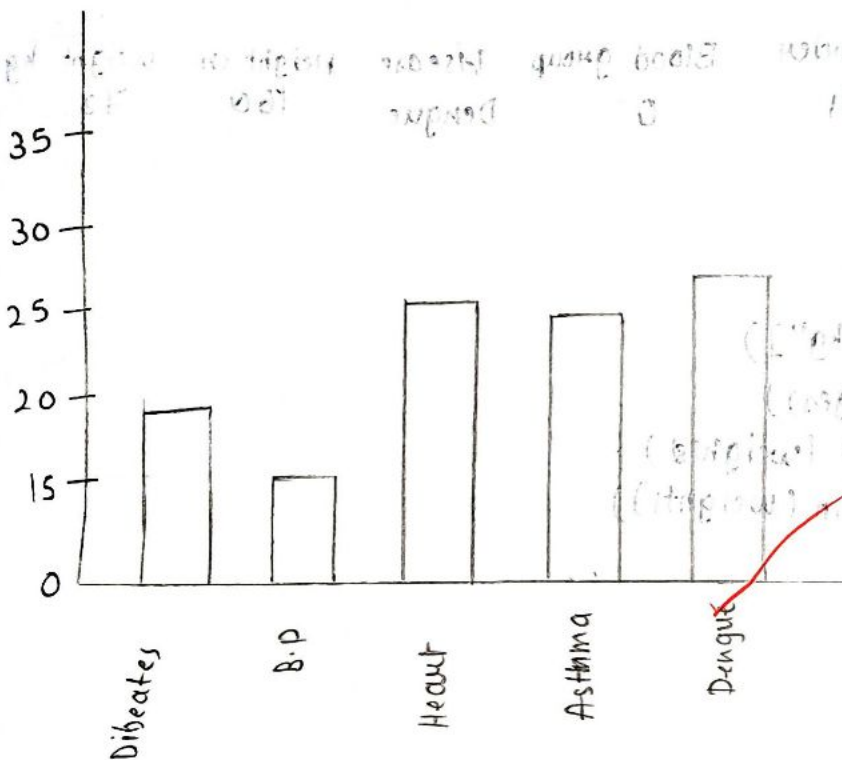
Max. weight: 70

Min weight: 55

Age vs weight (line graph)
 import matplotlib.pyplot as plt
 plt.plot(df["Age"], df["weight-kg"])
 plt.xlabel("Age")
 plt.ylabel("weight (kg)")
 plt.title("Age vs weight")
 plt.show()

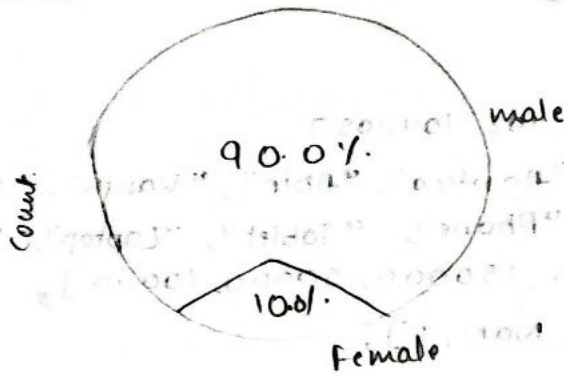


8) df.plot(x='Disease', y='age', kind='bar')



9) df. ["Gender"].value_counts().plot(kind="pie",)
 plt.title("Gender Distribution")
 plt.show()

Gender Distribution



Gender	Count	Percentage
male	90	90.0%
female	10	10.0%

df["Gender"].value_counts()

male 90
 female 10

1. Create a dataset of sales columns such as customer-Id, name, product name, cost, quantity, mode of payment etc.

import pandas as pd

import numpy as np

data = {

"Customer-Id": [101, 102, 103, 104, 105],

"Name": ["Atharva", "Agastya", "Abir", "Vansh", "Vanshika"],

"product": ["Laptop", "Phone", "Tablet", "Laptop", "Monitor"],

"Cost": [50000, 20000, 150000, 50000, 10000],

"Quantity": [1, 2, 3, NaN, 1],

"Mode-of-payment": ["Card", "Cash", NaN, "Card", "UPI"]

}

df = pd.DataFrame(data)

print(df)

Output-

	Customer-Id	Name	Product	Cost	Quantity	Mode-of-payment
0	101	Atharva	Laptop	50000	1	Card
1	102	Agastya	Phone	20000	2	Cash
2	103	Abir	Tablet	150000	3	NaN
3	104	Vansh	Laptop	50000	NaN	Card
4	105	Vanshika	Monitor	10000	1	UPI

select column-

df["Product"]

df[["Name", "Cost"]]

Output.

	Name	Cost
0	Atharva	50000
1	Agastya	20000
2	Abir	150000
3	Vansh	50000
4	Vanshika	10000

1) df.head()

2) df.describe()

3) df.info()

Output

1)

	customer-Id	Name	Product	cost	Quantity	mode- of payment
0	101	Atharva	Laptop	50000	1	card
1	102	Agastya	phon	20000	2	cash
2	103	Abir	Tablet	15000	3	NaN
3	104	Vaushit	Laptop	50000	NaN	card
4	105	Vaushika	Monitor	100000	1	UPI

2)

	customer-Id	cost	Quantity
count	5.000000	5.00000	4.000000
mean	103.000000	29000.000	1.730000
std	1.581139	19493.500	0.957427
min	101.000000	10000.000	1.000000

3) <class 'Panda.core.frame.DataFrame'>

Range Index 5 entries 0 to 4

Data columns (total 6 columns):

	column	count	Dtype
0	customer-Id	5	int64
1	Name	5	object
2	Product	4	object
3	cost	5	int64
4	Quantity	5	float64
5	mode-of-payment	4	object

df["Name"].fillna("unknown", inplace=True)

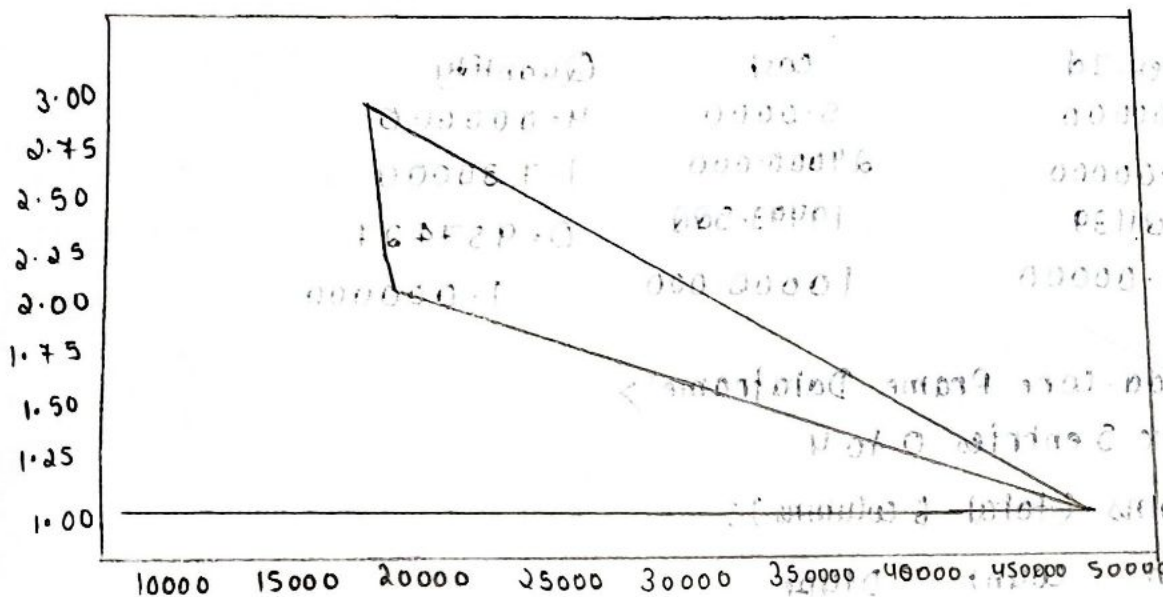
df["Quantity"].fillna(1, inplace=True)

df["mode-of-payment"].fillna("cash", inplace=True).

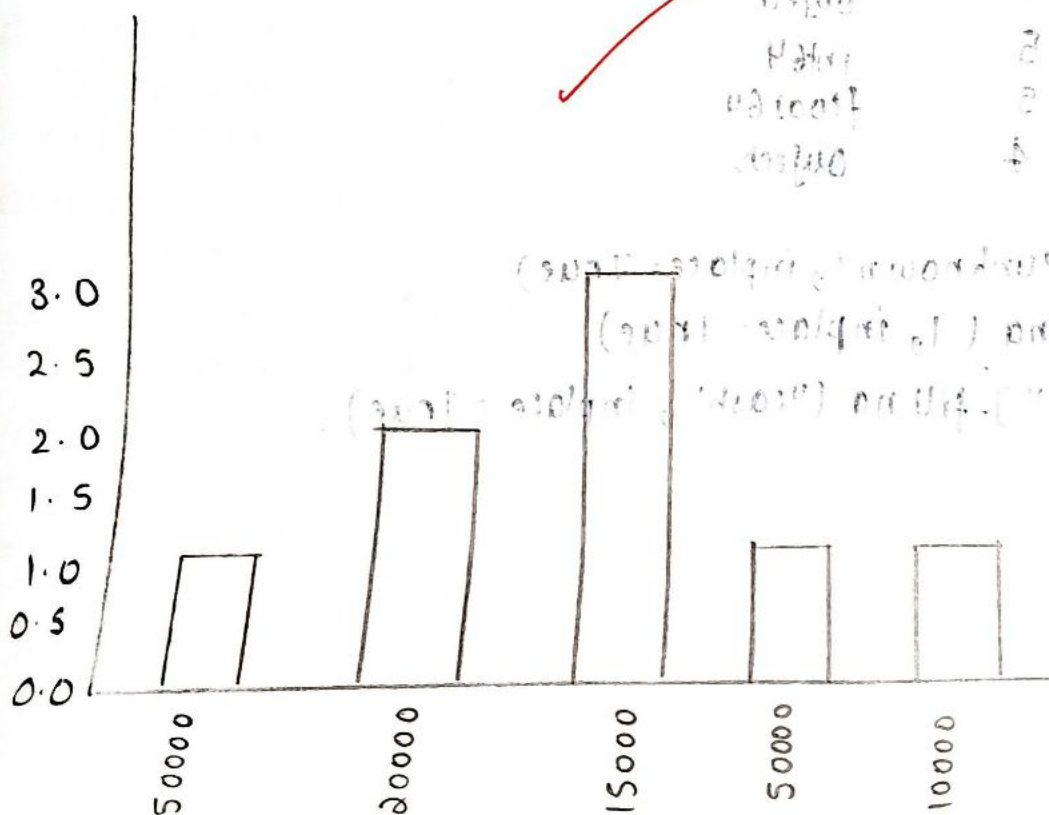
In put -

Customer-Id	Name	Product	Cost	Quantity	Mode-of-payment
101	Atharva	Laptop	50000	1	Card
102	Agastya	phon	20000	2	Cash
103	Abir	Tabler	15000	3	Cash
104	Vansh	Laptop	50000	1	Cash
105	Vanshika	Monitor	10000	1	UPI

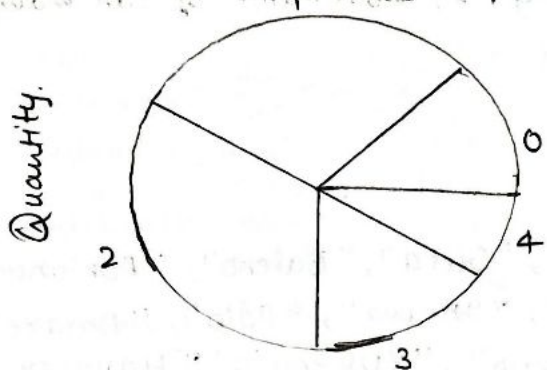
df.plot(x="cost", y="Quantity", kind="Line")



df.plot(x="cost", y="Quantity", kind="bar")



df.plot(x="cost", y="Quantity", kind="pie")



30/01/25

1. a dataset contains which contain information related to car
manufacture. 1) carname/ 2) car-type 3) color 4) price 5) mileage
6) no-of-seats, 7) engine^{mode} capacity, 8) mode name of car manufacturer.

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt.

data = {

"car-model": ["swift", "City", "Creta", "Baleno", "Fortuner", "i20",
"Verna", "seltos", "Innova", "Polo", "Amaze",
"Kushaq", "Nexon", "XUV300", "Harrier",
"Altaz", "Scorpio", "Compass", "Tiago", "Civic"],

"Type": ["Hatchback", "Sedan", "SUV", "Hatchback", "SUV",
"Hatchback", "Sedan", "SUV", "MPV", "Hatchback",
"Sedan", "SUV", "SUV", "SUV", "Hatchback",
"SUV", "SUV", "Hatchback", "Sedan"],

"Color": ["Red", "White", "Black", "Blue", "White", "Grey",
"Black", "Red", "Silver", "Blue", "White", "nan",
"Grey", "Black", "nan", "Red", "White",
"Black", "Blue", "nan"],

"Price-lakh": [6.5, 11.2, 15.8, 7.1, 32.0, 8.4, 10.9, 14.5, 21.0,
6.9, 7.5, nan, 13.2, 12.0, 18.5, nan, 17.8, 22.3,
5.9, 19.07]

"mileage-kmp": [22, 17, 16, 23, 12, 20, 18, nan, 14, 21, 18, 17, nan,
19, 15, 23, 14, nan, 24, 16],

"seats": [5, 5, 5, 5, 7, 5, 5, 5, 7, 5, 5, 5, nan, 5, 5, 5, 7, 5,
5, nan],

"engine-cc": [1200, 1500, 1500, 1200, 2000, 1200, 1500, 1400, 2400,
1000, 1200, 1500, 1200, nan, 2000, 1200, 2200, 2000,
1000, nan],

"manufacturer": ["Maruti", "Honda", "Hyundai", "Maruti", "Toyota",
"Hyundai", "Hyundai", "Kia", "Toyota", "Volkswagen",
"Honda", "Skoda", "Tata", "Mahindra", "Tata",
"Tata", "Mahindra", "Tata", "Tata"]

df = pd.DataFrame(data)

print(df)

Output

Car Model	Type	Color	Price-lakh	Milage-kmp	seats	Engine cc	Manufacturer
Swift	Hatchback	Red	6.5	22	5	1200	Maruti
City	Sedan	White	11.2	17	5	1500	Honda
Creta	SUV	Black	15.8	16	5	1500	Hyundai
Baleno	Hatchback	Blue	7.1	23	5	1200	Maruti
Fortuner	SUV	White	32.2	12	7	2800	Totota
I20	Hatchback	Grey	8.4	20	5	1200	Hyundai
Verna	Sedan	Black	10.9	18	5	1500	Hyundai
Seltos	SUV	Red	11.5	NAN	5	1500	Kia
Innova	MPV	Silver	21	14	7	2400	Toyota
Polo	Hatchback	Blue	6.9	21	5	1000	Volkswagen
Amaze	Sedan	White	7.5	18	5	1200	Honda
Kushaq	SUV	Nan	NAN	17	NAN	1500	Skoda
Nexon	SUV	Grey	13.2	NAN	5	1200	Tata
XUV300	SUV	Black	12	19	5	NAN	Marindra
Harrier	SUV	NAN	18.5	15	5	2000	Tata
Altos	Hatchback	Red	NAN	23	7	2200	Tata
Scorpio	SUV	White	17.8	14	5	1200	Marindra
Compass	SUV	Black	22.3	NAN	5	2000	Jeep
Tiago	Hatchback	Blue	5.9	24	5	1000	Tata
Civic	Sedan	Nan	19	16	NAN	NAN	Honda

df.isnull().sum()

Output-

car-model 0

Type 0

Color 3

Price-lakh 2

Milage-kmp 3

seats 2

Engine 2

manufacturer 0

dtype int64


```

1. df["Price-lakh"].fillna(df["Price-lakh"].mean(), inplace=True)
2. df["milage"].fillna(df["milage"].mean(), inplace=True)
3. df["seats"].fillna(df["seats"].mode()[0], inplace=True)
4. df["engine-cc"].fillna(df["engine-cc"].median(), inplace=True)
5. df["colour"].fillna("unknown", inplace=True)

```

Output -

Car model	Color	Price	milage	seats	Engine-cc
Swift	Red	6.5	22	5	1200
City	White	11.2	17	5	1500
Creta	Black	15.8	16	5	1500
Baleno	Blue	7.1	23	7	1200
Fortuner	White	32	12	5	2800
I20	Grey	8.4	20	5	1200
Verna	Black	10.5	18	5	1300
Seltos	Red	14.5	18	7	1400
Innova	Silver	21	14	5	1400
Polo	Blue	21	21	5	2400
Amaze	White	6.9	18	5	1000
		7.5	17	5	1200
Kushaq	Unknown	14.8	17	5	1500
Nexon	Grey	13.2	17	5	1500
		12	19	5	1200
KUV 300	Black	18.5	15	5	1350
Harrier	Unknown	14.89	23	7	2000
Altoz	Red	17.8	14	5	1200
Scorpio	White	22.3	18	5	2200
Compass	Black	5.9	24	5	2000
Tiago	Blue		16		1000
Civic	Unknown	19			1350


```

price_array = np.array(df["Price-lakh"])
print("Mean Price:", np.mean(price_array))
print("Max Price:", np.max(price_array))
print("Min Price:", np.min(price_array))
print("Standard deviation:", np.std(price_array))

```

Output-

mean price: 13.9166

Max price: 32.0

Min price: 5.9

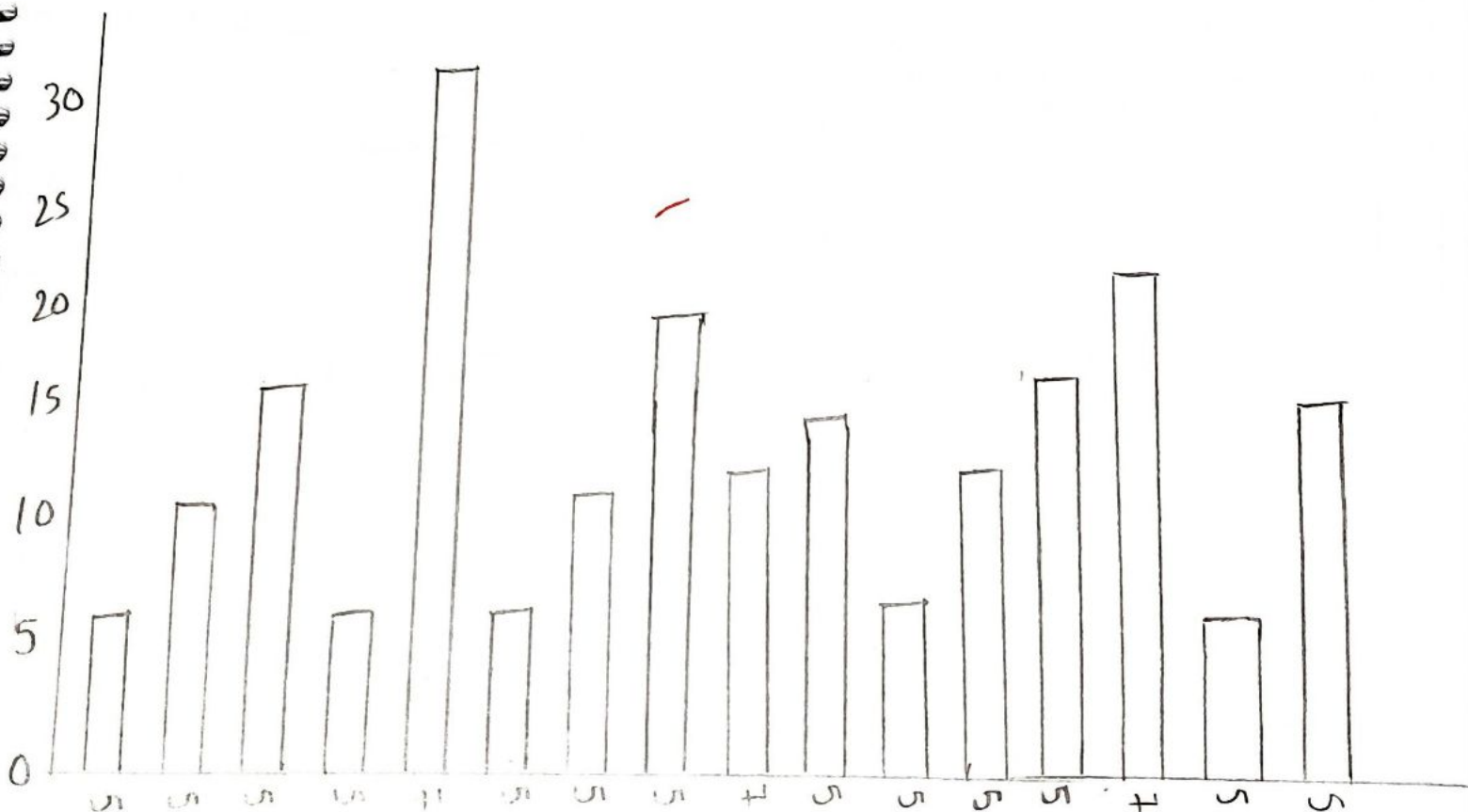
Standard deviation: 6.40517.

df.describe()

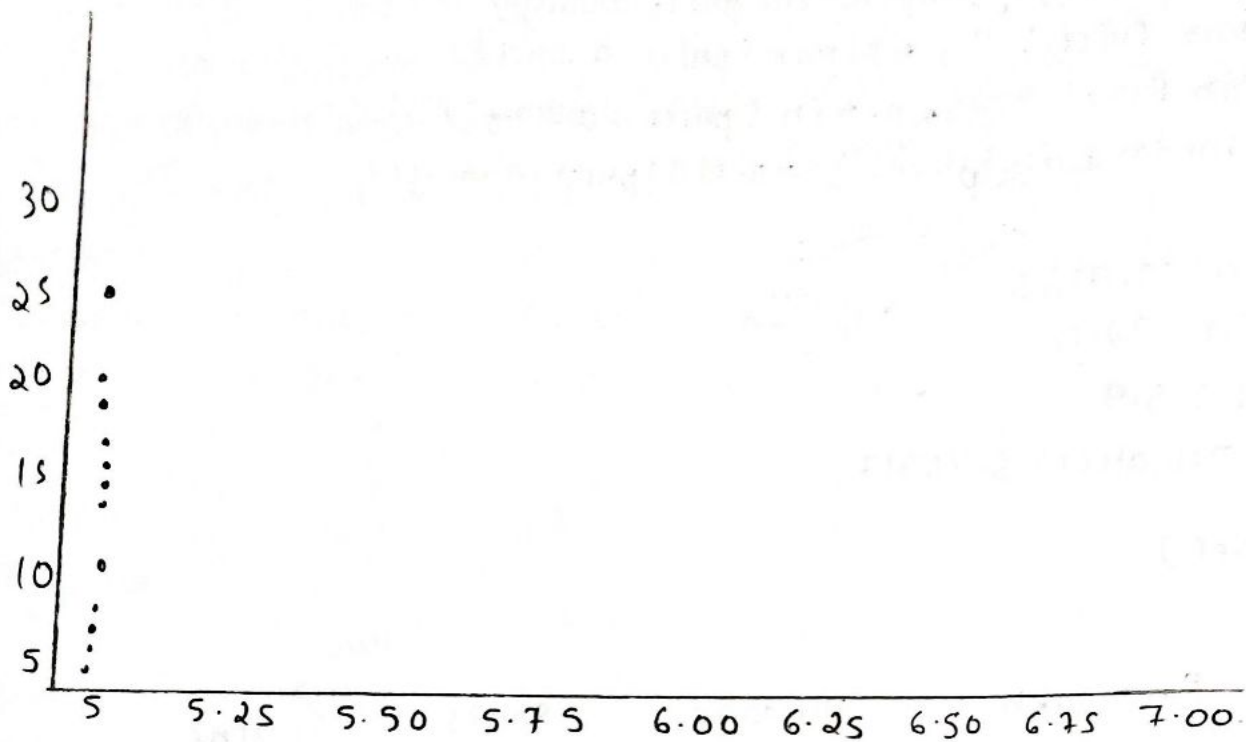
Output-

	Price lakh	Mileage	Seats	Engine
Count	20.000	20.000	20.0000	20.0000
mean	13.91667	18.176	5.3000	1545.000
std	6.5715	3.2644	0.73200	486.907
min	5.90000	12.0000	5.0000	1000.000
max	32.000	24.0000	7.0000	2800.0000

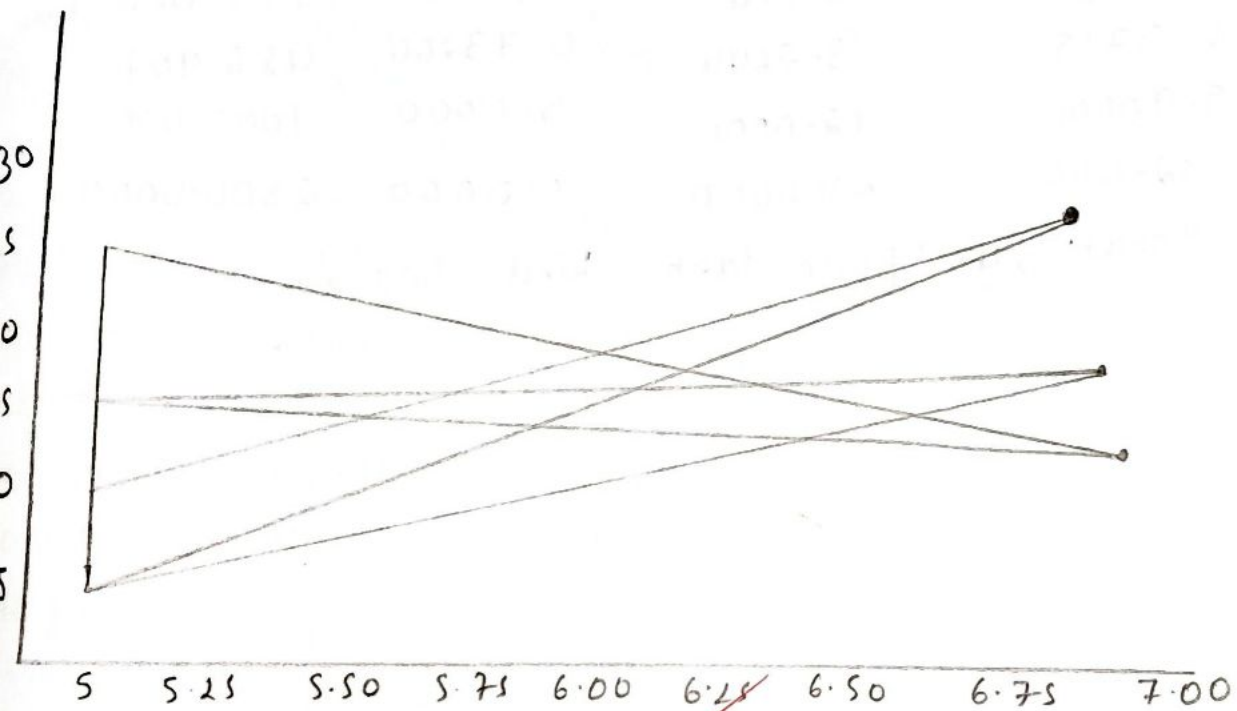
df.plot(x="Seats", y="Price-lakh", kind="Bar").



df.plot(x="seats", y="Price-lakh", kind="scatter").

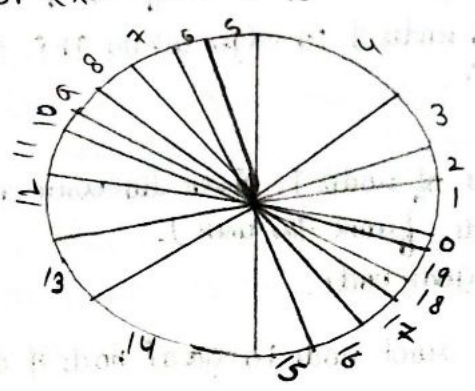


df.plot(x="seats", y="Price-lakh", kind="line").



py df.plot(x="seats", y="price-lakh", kind="pie")

price-lakh



B
08/04/26

Q: Design & develop a program to implement dfs algo for graph traversal.

1) Construct a static graph containing 6 nodes & 10 edges where it is stored in a form of dictionaries. The graph given is.

2) Create an another graph where the value of nodes is given by user at run time.
(Input the root node & the goal node from the user).

3) Display the path from initial node to goal node.

Y. i) Perform BFS traversal starting from root node to goal node & display the order of traversal.

ii) Display the shortest path from root node to goal node.

Al. graph = {
 "A": ["C", "B"],
 "B": ["D", "E", "F"],
 "C": ["F", "D"],
 "D": ["E", "F"],
 "E": ["F"],
 "F": []
}

visited = set()

```
def dfs(graph, Node, visited):  
    if Node not in visited:  
        visited.add(Node)  
        print(visited)  
        for n in graph[Node]:  
            dfs(graph, n, visited)
```

~~dfs(graph, "A", visited)~~

put :- { 'A' }
 { 'A', 'C' }
 { 'A', 'C', 'F' }
 { 'A', 'C', 'D', 'F' }
 { 'A', 'D', 'C', 'F', 'E' }
 { 'A', 'B', 'D', 'C', 'F', 'E' }

```

#2. n = int(input("Enter number of nodes:"))
graph = {}
for i in range(n):
    node = input("Enter node:")
    neighbors = input("Enter neighbors separated by space:").split()
    graph[node] = neighbors

root_node = input("Enter the root Node:")
goal_node = input("Enter the goal Node:")
visited = set()

def dfs(graph, node, goal_node):
    if node in visited:
        return False

    print(node, end=" ")
    visited.add(node)

    if node == goal_node:
        print(f"\n Goal node found - {goal_node}")

    for neighbor in graph[node]:
        if dfs(graph, neighbor, goal_node):
            return True

    return False

print("DFS traversal order:-")
dfs(graph, root_node, goal_node)

```

Output- Enter number of nodes: 6
Enter node: A
Enter neighbors separated by space: CB
Enter node: B
Enter neighbors separated by space: DE
Enter node: C
Enter neighbors separated by space: FD
Enter node: D
Enter neighbors separated by space: E
Enter node: E
Enter neighbors separated by space: F
Enter node: F
Enter neighbors separated by space: D

Enter the root Node: A

Enter the Goal Node: E

BFS Traversal Order:-

A C A D E

Goal node found - E

True.

A3. graph = {
 "A": ["C", "B"],
 "B": ["D", "E", "F"],
 "C": ["F", "D"],
 "D": ["E", "F"],
 "E": ["F"],
 "F": []
}

```
def bfs (graph, Node):  
    visited = []  
    queue = []  
    visited.append (Node)  
    queue.append (Node)  
    while queue:  
        m = queue.pop (0)  
        print (m)  
        for n in graph [m]:  
            if n not in visited:  
                visited.append (n)  
                queue.append (n)
```

bfs (graph, "A")

Output :-
A
C
B
F
D
E


```

AV. n = int(input("Enter number of nodes: "))
graph = {}

for i in range(n):
    node = input("Enter node: ")
    neighbors = input("Enter neighbors separated by space: ").split()

Root-node = input("Enter the Root Node: ")
Goal-node = input("Enter the Goal Node: ")

print(graph)

def bfs(graph, Node):
    visited = []
    queue = []
    visited.append(Node)
    queue.append(Node)
    while queue:
        m = queue.pop(0)
        print(m, end=" ")
        if m == Goal-node:
            print(f"\n Goal node found - {Goal-node}")
            return
        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)
    print("\n Goal node not found")

print("BFS Traversal Order:")
bfs(graph, Root-node)

```

Output :

Enter number of nodes: 4
Enter node: A
Enter neighbors separated by space: C B
Enter node: B
Enter neighbors separated by space: C
Enter node: C
Enter neighbors separated by space: D
Enter node: D
Enter neighbors separated by space:
Enter the Root Node: A
Enter the Goal Node: C

{ 'A': ['C', 'B'], 'B': ['C'], 'C': ['D'], 'D': [] }

BFS Traversal Order:

A C

Goal Node found - C.

A4 ii) graph = {

"A": ["C", "B"],

"B": ["D", "E", "F"],

"C": ["F", "D"],

"D": ["E", "F"],

"E": ["F"],

"F": [] }

start = input("Enter start Node: ")

goal = input("Enter Goal Node: ")

def bfs_path(graph, start, goal):

visited = []

queue = []

parent_node = {}

visited.append(start)

queue.append(start)

parent_node[start] = None

while queue:

current = queue.pop(0)

if current == goal:

path = []

while current is not ~~None~~ None:

path.append(current)

current = parent_node[current]

path.reverse()

print("Shortest Path:", path)

return

for neighbour in graph[current]:

if neighbour not in visited:

visited.append(neighbour)

queue.append(neighbour)

parent_node[neighbour] = current.

print ("No path found")

bfs-path (graph, start, goal)

Output:- Enter start Node: A

Enter Goal Node: D

Shortest Path: ['A', 'C', 'D']

Path	Distance	Node	Cost	Order
1	00000000	Start	000	1
2	00000000	Start	000	2
3	00000000	Start	000	3
4	00000000	Start	000	4
5	00000000	Start	000	5
6	00000000	Start	000	6
7	00000000	Start	000	7
8	00000000	Start	000	8
9	00000000	Start	000	9
10	00000000	Start	000	10
11	00000000	Start	000	11
12	00000000	Start	000	12
13	00000000	Start	000	13
14	00000000	Start	000	14
15	00000000	Start	000	15
16	00000000	Start	000	16
17	00000000	Start	000	17
18	00000000	Start	000	18
19	00000000	Start	000	19
20	00000000	Start	000	20
21	00000000	Start	000	21
22	00000000	Start	000	22
23	00000000	Start	000	23
24	00000000	Start	000	24
25	00000000	Start	000	25
26	00000000	Start	000	26
27	00000000	Start	000	27
28	00000000	Start	000	28
29	00000000	Start	000	29
30	00000000	Start	000	30
31	00000000	Start	000	31
32	00000000	Start	000	32
33	00000000	Start	000	33
34	00000000	Start	000	34
35	00000000	Start	000	35
36	00000000	Start	000	36
37	00000000	Start	000	37
38	00000000	Start	000	38
39	00000000	Start	000	39
40	00000000	Start	000	40
41	00000000	Start	000	41
42	00000000	Start	000	42
43	00000000	Start	000	43
44	00000000	Start	000	44
45	00000000	Start	000	45
46	00000000	Start	000	46
47	00000000	Start	000	47
48	00000000	Start	000	48
49	00000000	Start	000	49
50	00000000	Start	000	50
51	00000000	Start	000	51
52	00000000	Start	000	52
53	00000000	Start	000	53
54	00000000	Start	000	54
55	00000000	Start	000	55
56	00000000	Start	000	56
57	00000000	Start	000	57
58	00000000	Start	000	58
59	00000000	Start	000	59
60	00000000	Start	000	60
61	00000000	Start	000	61
62	00000000	Start	000	62
63	00000000	Start	000	63
64	00000000	Start	000	64
65	00000000	Start	000	65
66	00000000	Start	000	66
67	00000000	Start	000	67
68	00000000	Start	000	68
69	00000000	Start	000	69
70	00000000	Start	000	70
71	00000000	Start	000	71
72	00000000	Start	000	72
73	00000000	Start	000	73
74	00000000	Start	000	74
75	00000000	Start	000	75
76	00000000	Start	000	76
77	00000000	Start	000	77
78	00000000	Start	000	78
79	00000000	Start	000	79
80	00000000	Start	000	80
81	00000000	Start	000	81
82	00000000	Start	000	82
83	00000000	Start	000	83
84	00000000	Start	000	84
85	00000000	Start	000	85
86	00000000	Start	000	86
87	00000000	Start	000	87
88	00000000	Start	000	88
89	00000000	Start	000	89
90	00000000	Start	000	90
91	00000000	Start	000	91
92	00000000	Start	000	92
93	00000000	Start	000	93
94	00000000	Start	000	94
95	00000000	Start	000	95
96	00000000	Start	000	96
97	00000000	Start	000	97
98	00000000	Start	000	98
99	00000000	Start	000	99
100	00000000	Start	000	100

Q% Create a dataset customer or person is going to purchase, house, flat. Base on certain requirement such as type of house, area of residence, locality of residence, purchasing cost, no. of floors etc. Normalise cost & Area. Applying min/max.

Q% Create a dataset to medical (healthcare) which has columns such as pId, Name, Diagnosis, sugar level, BP, annual income, Age.
df = pd.DataFrame(data).

AI.	Type (BHK)	Area (sq. ft)	locality	Purchasing-cost	Floors
0	1	550	Dehradun	2500000	1
1	2	850	Lucknow	4500000	2
2	3	1200	Haridwar	7500000	3
3	2	950	Kanpur	5000000	2
4	3	1500	Noida	9000000	4
5	1	600	Nainital	2800000	1
6	2	1100	Dehradun	6500000	3
7	4	2000	Noida	14000000	5
8	3	1700	Haridwar	11000000	4
9	2	2200	Nainital	16000000	5
10	1	1300	Kanpur	8000000	3
11	4	500	Dehradun	2200000	1
12	3	900	Noida	4800000	2
13	2	1400	Haridwar	8500000	3
14	3	2500	Lucknow	20000000	6
15	4	1050	Kanpur	6000000	2
16	2	650	Nainital	3000000	1
17	1	1600	Dehradun	10000000	4
18	3	2300	Noida	18000000	5
19	4				

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
data = {
```

```
    "Type (BHK)": [1, 2, 3, 2, 3, 1, 2, 4, 3, 2, 1, 4, 3, 2, 3, 4, 2, 1, 3, 4],
```

```
    "Area (sq. ft)": [550, 850, 1200, 950, 1500, 600, 1100, 2000, 1700,  
                     1000, 500, 2200, 1300, 900, 1400, 2500, 1050, 650,  
                     1600, 2300],
```

```
    "Locality": ["Dehradun", "Lucknow", "Haridwar", "Kanpur", "Noida",  
                "Nainital", "Dehradun", "Noida", "Haridwar", "Lucknow",  
                "Dehradun", "Nainital", "Kanpur", "Noida"],
```

```
    "purchasing-cost": [2500000, 4500000, 7500000, 5000000, 9000000,  
                        2800000, 6500000, 14000000, 11000000, 5500000,  
                        2200000, 16000000, 8000000, 4800000, 8500000,  
                        2000000, 6000000, 3000000, 10000000,  
                        18000000],
```

```
    "Floors": [1, 2, 3, 2, 4, 1, 3, 5, 4, 2, 1, 5, 3, 2, 3, 6, 2, 1, 4, 5]
```

```
* plt.scatter(df["Area (sq. ft)"], df["purchasing-cost"])
```

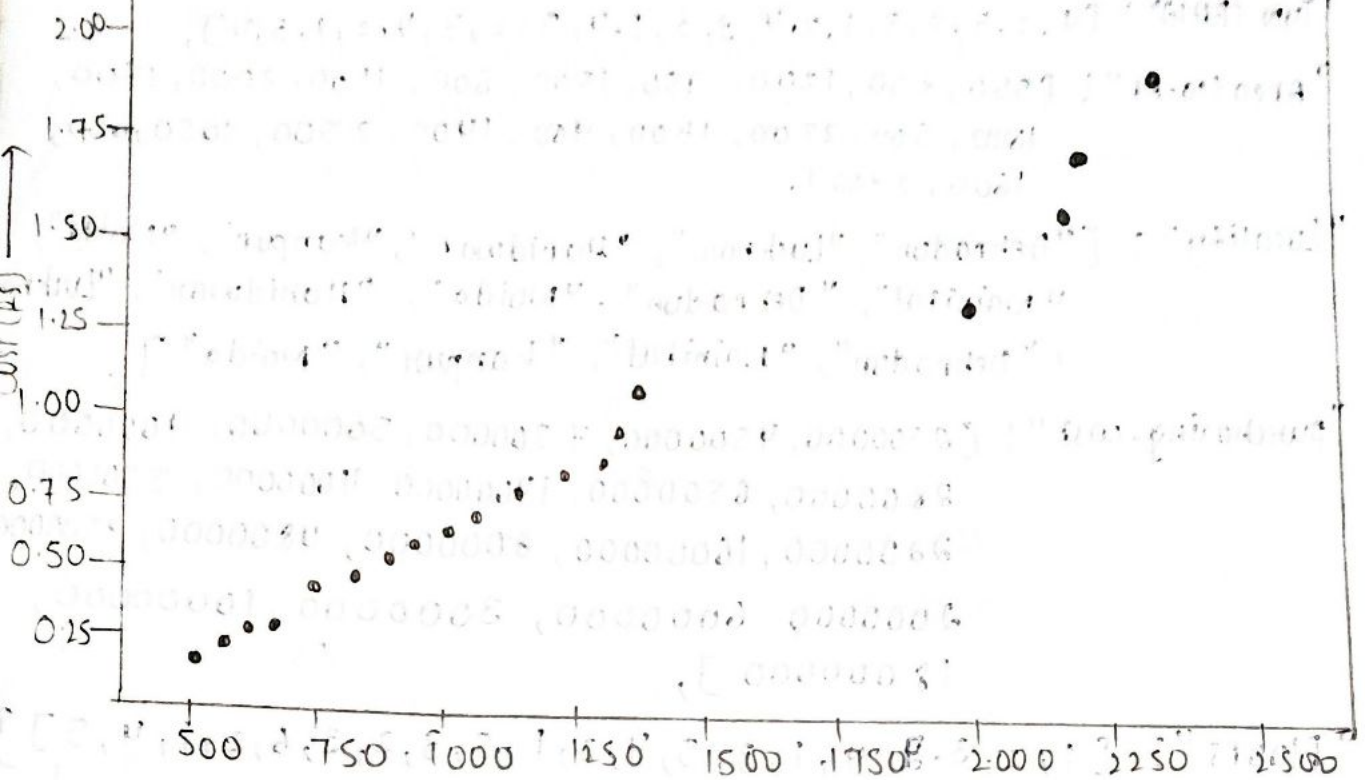
```
plt.title("Before Normalisation")
```

```
plt.xlabel("Area (sq. ft)")
```

```
plt.ylabel("Cost (Rs)")
```

```
plt.show()
```

Before Normalisation



Area (sq.ft)

$$\text{area_min} = \text{df}["\text{Area (sq.ft)}"].\text{min}()$$

$$\text{area_max} = \text{df}["\text{Area (sq.ft)}"].\text{max}()$$

$$\text{df}["\text{After-Normalized}"] = (\text{df}["\text{Area (sq.ft)}"] - \text{area_min}) / (\text{area_max} - \text{area_min})$$

$$\text{cost_min} = \text{df}["\text{Purchasing - cost}"].\text{min}()$$

$$\text{cost_max} = \text{df}["\text{Purchasing - cost}"].\text{max}()$$

$$\text{df}["\text{Purchasing - cost-Normalized}"] = (\text{df}["\text{Purchasing - cost}"] - \text{cost_min}) / (\text{cost_max} - \text{cost_min})$$



	Type (BHK)	Area (sqft)	Locality	Purchasing cost	Floors	Area Normalised	Purchasing cost Normalised
0	1	550	Dehradun	2500000	1	0.025	0.016854
1	2	850	Lucknow	4500000	2	0.175	0.129213
2	3	1200	Hawidwar	7500000	3	0.350	0.297753
3	2	950	Kanpur	5000000	2	0.225	0.157303
4	3	1500	Noida	90000000	4	0.500	0.382022
5	1	600	Nainital	2800000	1	0.050	0.033708
6	2	1100	Dehradun	6500000	3	0.300	0.241573
7	4	2000	Noida	14000000	5	0.750	0.662921
8	3	1700	Hawidwar	11000000	4	0.600	0.494382
9	2	1000	Lucknow	5500000	2	0.250	0.185393
10	1	500	Dehradun	2200000	1	0.000	0.000000
11	4	2200	Nainital	1600000	5	0.850	0.775281
12	3	1300	Kanpur	8000000	3	0.400	0.325843
13	2	900	Noida	4800000	2	0.200	0.146061
14	3	1400	Hawidwar	8500000	3	0.450	0.353931
15	4	2500	Lucknow	20000000	6	1.000	1.000000
16	2	1050	Kanpur	6000000	2	0.275	0.213481
17	1	650	Nainital	3000000	1	0.075	0.044944
18	3	1600	Dehradun	10000000	4	0.550	0.438211
19	4	2300	Noida	18000000	3	0.900	0.887611

* sns.scatterplot(x=df["Area-Normalised"], y=df["Purchasing cost-Normalised"])

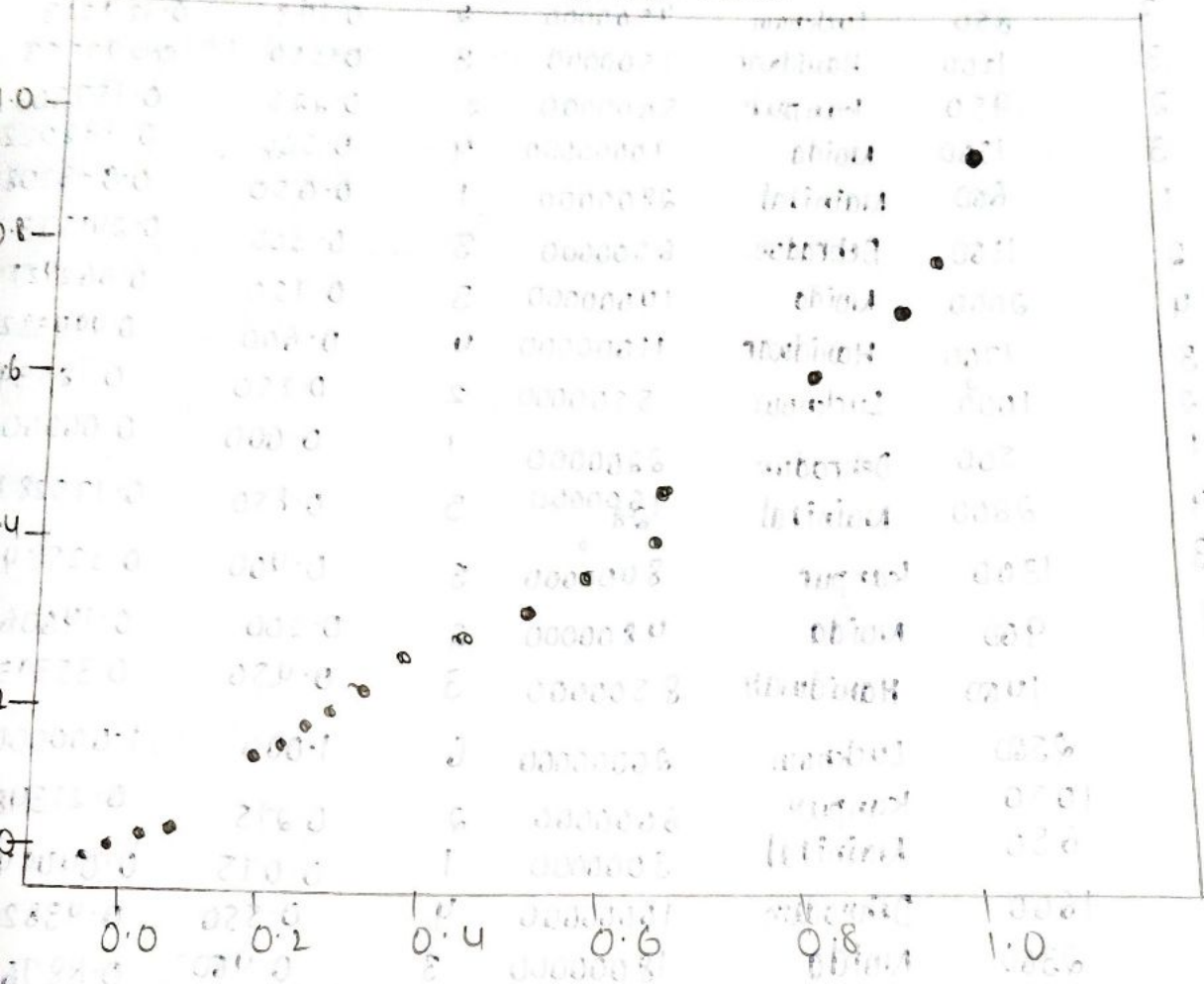
plt.title("After Min-Max Normalisation")

plt.xlabel("Normalised Area")

plt.ylabel("Normalised cost")

plt.show()

After min-max Normalization



Normalized Area →

you are provided" (p. 5) "The first three are the most important" (p. 6)

(The first three are the most important)

(The first three are the most important)

(The first three are the most important)

(The first three are the most important)

A2. data = {

"P Id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

"Name": ["Amit", "Neha", "Rahul", "Sneha", "Arjun", "Priya", "Karan",
"Meera", "Rohit", "Anjali"],

"Diagnosis": ["Diabetes", "Hypertension", "Normal", "Diabetes", "Normal",
"Hypertension", "Diabetes", "Normal", "Hypertension",
"Normal"],

"Sugar-level": [180, 95, 85, 200, 90, 100, 210, 88, 110, 92],

"BP": [140, 150, 120, 160, 110, 145, 170, 115, 155, 118],

"Annual-income": [300000, 500000, 250000, 450000, 350000,
600000, 700000, 320000, 480000, 290000],

"Age": [45, 50, 30, 55, 28, 48, 60, 35, 52, 32].

}

df = pd.DataFrame(data)

df.

	P Id	Name	Diagnosis	Sugar-level	BP	Annual-income	Age
0	1	Amit	Diabetes	180	140	300000	45
1	2	Neha	Hypertension	95	150	500000	50
2	3	Rahul	Normal	85	120	250000	30
3	4	Sneha	Diabetes	200	160	450000	55
4	5	Arjun	Normal	90	110	350000	28
5	6	Priya	Hypertension	100	145	600000	48
6	7	Karan	Diabetes	210	170	700000	60
7	8	Meera	Normal	88	115	320000	35
8	9	Rohit	Hypertension	110	155	480000	52
9	10	Anjali	Normal	92	118	290000	32

* plt.scatter(df["Annual Income"], df["BP"])

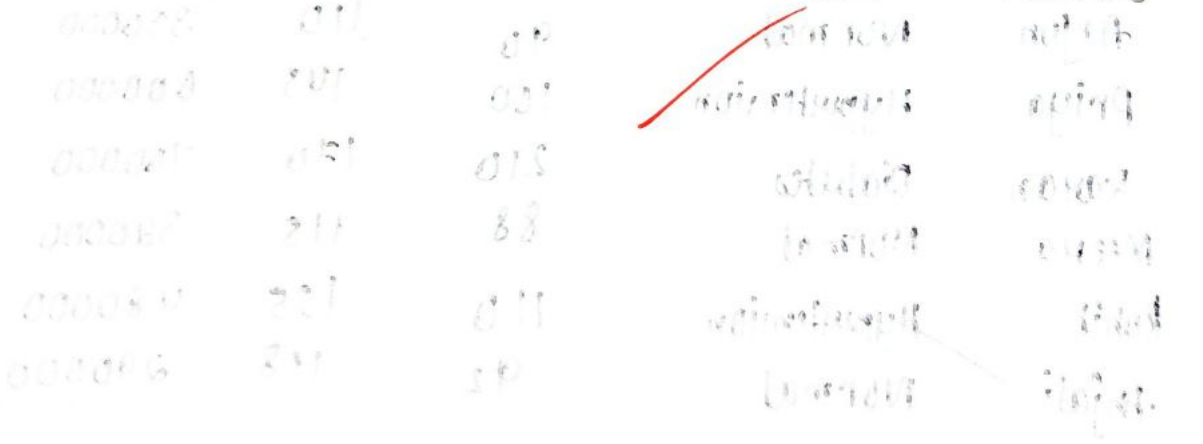
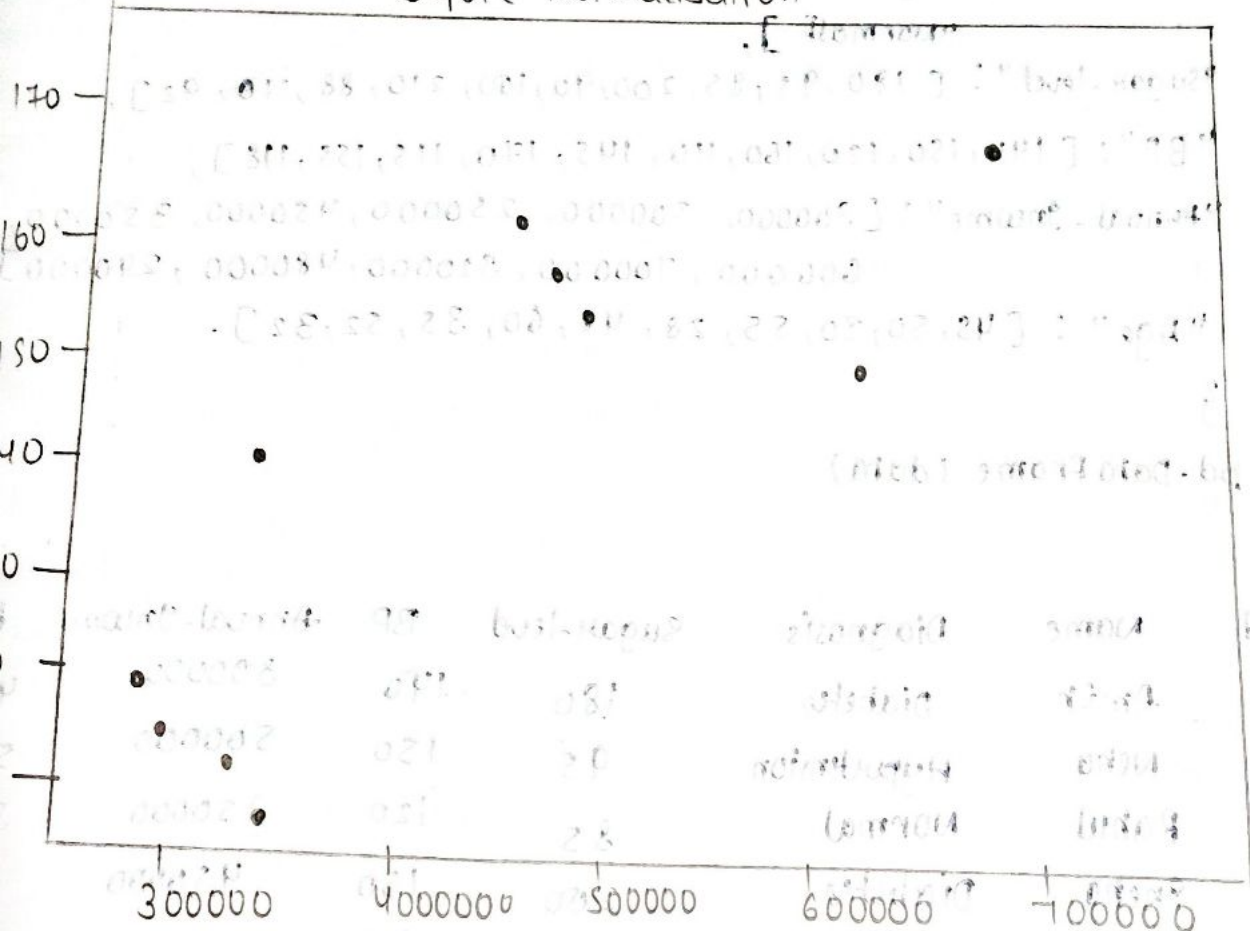
plt.title("Before Normalisation")

plt.xlabel("Annual Income")

plt.ylabel("BP")

plt.show()

Before Normalisation



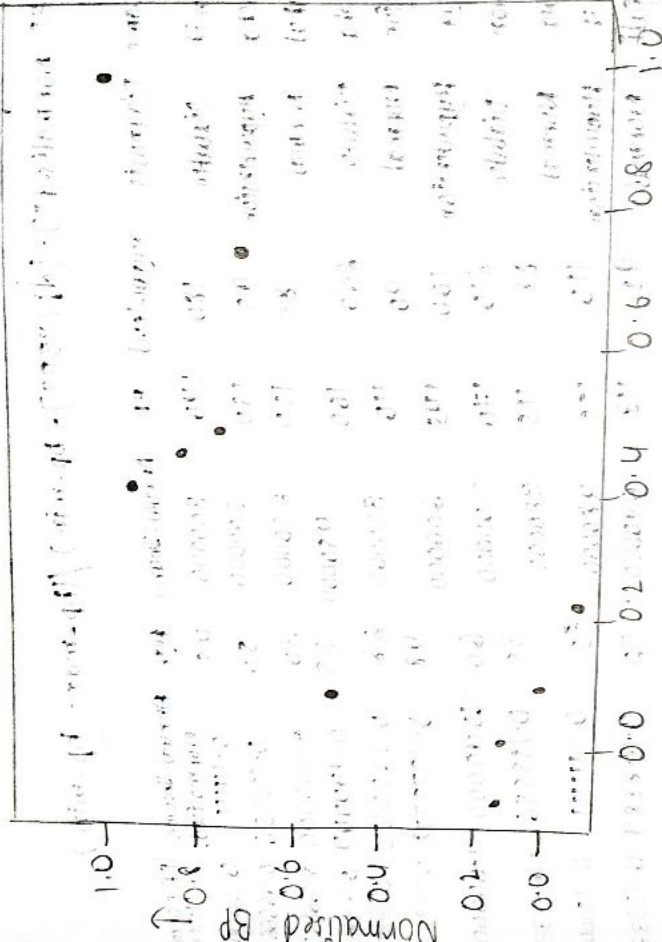
* $\text{income_min} = \text{df}["\text{Annual-Income}"].\text{min}()$
 $\text{income_max} = \text{df}["\text{Annual-Income}"].\text{max}()$
 $\text{df}["\text{Annual-Income-Normalised}"] = ((\text{df}["\text{Annual-Income}"] - \text{income_min}) / (\text{income_max} - \text{income_min}))$
 $\text{bp_min} = \text{df}["\text{BP}"].\text{min}()$
 $\text{bp_max} = \text{df}["\text{BP}"].\text{max}()$
 $\text{df}["\text{BP-Normalised}"] = (\text{df}["\text{BP}"] - \text{bp_min}) / (\text{bp_max} - \text{bp_min})$

PId	Name	Diagnosis	sugar-level	BP	Annual-Income	Age	Annual-Income Normalised.	BP-Normalised.
0	1	Amit	Diabetes	180	140	300000	45	0.111111
1	2	Neha	Hypertension	95	150	500000	50	0.555556
2	3	Rahul	Normal	85	120	250000	30	0.000000
3	4	Sneha	Diabetes	200	160	450000	55	0.444444
4	5	Arjun	Normal	90	110	350000	28	0.222222
5	6	Priya	Hypertension	100	145	600000	48	0.777778
6	7	Karan	Diabetes	210	170	700000	60	1.000000
7	8	Meera	Normal	88	115	320000	35	0.155556
8	9	Rohit	Hypertension	110	155	480000	52	0.511111
9	10	Anjali	Normal	92	118	290000	32	0.088889

* $\text{plt.scatter}(\text{df}["\text{Annual-Income-Normalised}"], \text{df}["\text{BP-Normalised}"])$
 $\text{plt.title}("\text{After-Normalisation}')$
 $\text{plt.xlabel}("\text{Normalised Annual Income}')$
 $\text{plt.ylabel}("\text{Normalised BP}')$
 $\text{plt.show}()$

Graph -

After Normalisation



Normalised Annual Income →



27/3/26

import pandas as pd.

import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

df = pd.read_csv('student-data.csv')

SNO	Study Hours	sleep	Marks
1	7	7	82.07
2	4	6	72.8
3	8	8	84.73
4	5	6	65.48
5	7	6	100.16
6	3	5	40.07
7	7	5	79.34
8	8	6	89.45
9	5	9	82.4
10	4	6	88.96
11	8	8	100.55
12	8	5	74.99
13	12	7	79.19
14	2	9	69.77
15	5	7	66.46
16	1	7	46.66
17	6	8	40.94
18	9	9	44.78
19	1	4	
20	3	4	

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=2)

model = LinearRegression()

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print(model.intercept)

→ 34.74432

```
print(model.coef_)  
→ [5.167 2.0275]
```

```
from sklearn.metrics import r2_score
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("R-squared", r2)
```

```
n = X_test.shape[0]
```

```
p = X_test.shape[1]
```

```
adjusted_r2 = 1 - ((1 - r2) * (n - 1) / (n - p - 1))
```

```
print("adjusted R-squared", adjusted_r2)
```

```
from sklearn.metrics import mean_absolute_error
```

```
from sklearn.metrics import mean_squared_error
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
print(mae)
```

```
print(mse)
```

```
→ 4181.194
```

```
→ 33596915.85
```